

Data Base Documentation

@Bia Mello

Cronograma do Banco de Dados (PostgreSQL)

Equipe: Beatriz Pimentel de Mello

Objetivo: Modelar, implementar e gerenciar o banco de dados PostgreSQL que armazenará métricas, logs, alertas e informações de hosts, garantindo performance, integridade e disponibilidade para os demais módulos do sistema.

Estrutura do Cronograma

Semana	Período	Foco Principal
1	21/04 – 27/04	Modelagem inicial e ambiente Docker
2	28/04 – 04/05	Integração com Backend e primeiras queries
3	05/05 – 11/05	Índices, performance e retenção de dados
4	12/05 – 18/05	Estrutura para logs e parsing
5	19/05 – 25/05	Tabela de alertas e suporte à detecção
6	26/05 – 01/06	Otimização e preparação para parcial
7	02/06 – 05/06	Backup, restore e documentação final

♦ Semana 1 | 21 – 27 abr

🎯 **Meta da Semana:** Ter o PostgreSQL rodando em Docker com o schema inicial definido e validado.

📋 Tarefas

1.1 Configurar o ambiente PostgreSQL com Docker

- Criar `docker-compose.yml` com serviço `db`:
- Imagem: `postgres:15`
- Variáveis de ambiente: `POSTGRES\_USER`, `POSTGRES\_PASSWORD`, `POSTGRES\_DB`
- Volume mapeado: `./data/postgres:/var/lib/postgresql/data` (persistência)
- Porta exposta: `5432` (apenas desenvolvimento)
- Executar `docker-compose up -d`

1.2 Modelagem Conceitual (DER)

- Identificar as entidades do minimundo: `host`, `agents`, `host\_discovery`, `metrics`, `logs`, `alerts`, `active\_connections`
- Definir os relacionamentos diretos com `host` (nenhum relacionamento direto entre as demais entidades):
- Desenhar o Diagrama Entidade-Relacionamento (DER) utilizando notação de Peter Chen (retângulos, losangos, cardinalidades).

```
1 26/04/2026
2 Modelagem conceitual concluída:
3   * `host` (1,1) – possui – (1,1) `agents`
4   * `host` (1,1) – descreve – (0,1) `host_discovery`
5   * `host` (1,1) – gera – (0,N) `metrics`
6   * `host` (1,1) – registra – (0,N) `logs`
7   * `host` (1,1) – dispara – (0,N) `alerts`
8   * `host` (1,1) – coleta – (0,N) `active_connections`
```

1.3 Modelagem Lógica (tabelas, atributos, chaves e normalização)

- Transformar o DER em um conjunto de tabelas (relações) com:
- Nomes das tabelas: `hosts`, `agents`, `host\_discovery`, `metrics`, `logs`, `alerts`, `active\_connections`
- Colunas, tipos de dados e restrições de integridade (PK, FK, NOT NULL, UNIQUE, DEFAULT)
- Aplicar a 1ª, 2ª e 3ª Formas Normais
- Documentar a escolha dos tipos (ex.: `DECIMAL(5,2)` para percentuais, `BIGINT` para tráfego de rede, `JSONB` para dados semi-estruturados)

```
1 26/04/2026
2 Modelagem lógica em andamento:
3 Criação de 4 tabelas de 7 (sem relacionamento definido ainda na modelagem)
4 * host
5 * agents
6 * host_discovery
7 * metrics
8
```

1.4 Criar script SQL inicial ('schema.sql') com todas as 7 tabelas

- Utilizar as definições da modelagem lógica para escrever os comandos 'CREATE TABLE'
- Incluir chaves primárias, estrangeiras (com 'ON DELETE CASCADE') e índices básicos

1.5 Testar a criação das tabelas

- Executar: 'docker exec -i monitor_db psql -U admin -d monitor_db < schema.sql'
- Verificar com '\dt' (devem aparecer as 7 relações)
- Inserir dados de teste simples para validar chaves estrangeiras e tipos

1.6 Documentar o esquema (DER e modelo lógico)

- Incluir o diagrama conceitual (DER) e as descrições das tabelas, colunas, tipos e relacionamentos
- Gerar um arquivo 'README_DB.md' para compartilhar com a equipe

✓ Entregáveis da Semana

- Container PostgreSQL rodando em Docker (volume persistente)
- Modelo Conceitual (DER) documentado
- Modelo Lógico (tabelas, atributos, chaves, normalização) documentado
- Script 'schema.sql' com as 7 tabelas criadas e validadas
- Documentação completa do esquema do banco de dados

♦ Semana 2 | 28 abr – 04 mai

🎯 **Meta da Semana:** Garantir que o Backend consiga inserir e consultar métricas.

📋 Tarefas

- **2.1** Criar usuário de aplicação com permissões limitadas (não usar 'postgres' superusuário no código Flask):

```
1 CREATE USER app_user WITH PASSWORD 'senha_segura';
2 GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO app_user;
3 GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO app_user;
4
```

- **2.2** Testar conexão do Backend (SQLAlchemy) com o banco usando as credenciais do `app_user`.
 - **2.3** Validar inserção de métricas via endpoint `POST /api/metrics` (acompanhar os logs do PostgreSQL com `docker logs -f db`).
 - **2.4** Ajudar Douglas/Fernando a escrever queries SQLAlchemy para:
 - Inserir ou obter `host_id` a partir do `hostname`.
 - Inserir métricas.
 - Consultar últimas N métricas por host.
 - **2.5** Executar consultas manuais para verificar os dados inseridos: `SELECT * FROM metrics ORDER BY timestamp DESC LIMIT 10;`
 - `**C$ROW3#W#ads#`
-

✓ Entregáveis da Semana

- Backend integrado com sucesso ao PostgreSQL.
 - Dados de métricas persistidos e consultáveis.
-

♦ Semana 3 | 05 – 11 mai

🎯 **Meta da Semana:** Melhorar performance com índices e implementar política de retenção.

📋 Tarefas

- **3.1** Criar índices para otimizar as consultas mais frequentes:
 - `CREATE INDEX idx_metrics_host_timestamp ON metrics(host_id, timestamp DESC);`
 - `CREATE INDEX idx_logs_host_timestamp ON logs(host_id, timestamp DESC);`
 - `CREATE INDEX idx_logs_parsed_ip ON logs((parsed_data->>'ip_origem'));` (índice GIN ou BTREE, dependendo da necessidade)
- **3.2** Analisar plano de execução com `EXPLAIN ANALYZE` para uma query típica de consulta de métricas.
- **3.3** Implementar política de retenção de dados:
 - Criar função SQL ou script Python (executado via cron dentro do container ou agendador externo) para deletar métricas com mais de **7 dias**:

```
1 | DELETE FROM metrics WHERE timestamp < NOW() - INTERVAL '7 days';  
2 |
```

- Para logs, manter também 7 dias (ajustável).
- **3.4** Testar o script de limpeza e medir impacto na performance.
- **3.5** Verificar se o auto-vacuum do PostgreSQL está ativo e configurado adequadamente.

✓ Entregáveis da Semana

- Índices criados e performance validada.
- Rotina de limpeza de dados antigos implementada.

♦ Semana 4 | 12 – 18 mai

🎯 **Meta da Semana:** Preparar estrutura para logs e facilitar o parsing no backend.

📋 Tarefas

- **4.1** Revisar a tabela `logs` : campo `parsed_data JSONB` está adequado.
- **4.2** Auxiliar o Backend a escrever query para contar tentativas de login falhas por IP nos últimos N minutos:

```
1 SELECT parsed_data->>'ip_origem' AS ip, COUNT(*)
2 FROM logs
3 WHERE log_type = 'auth'
4       AND parsed_data->>'status' = 'failed'
5       AND timestamp > NOW() - INTERVAL '1 minute'
6 GROUP BY ip
7 HAVING COUNT(*) >= 5;
8
```

- **4.3** Criar uma **View** para facilitar consultas de eventos de segurança:

```
1 CREATE VIEW failed_logins_last_hour AS
2 SELECT host_id, parsed_data->>'ip_origem' AS ip, timestamp
3 FROM logs
4 WHERE log_type = 'auth' AND parsed_data->>'status' = 'failed'
5       AND timestamp > NOW() - INTERVAL '1 hour';
6
```

- **4.4** Testar a view com dados de exemplo.

✓ Entregáveis da Semana

- Estrutura de logs otimizada para consultas de segurança.
- Views auxiliares criadas.

♦ Semana 5 | 19 – 25 mai

🎯 **Meta da Semana:** Implementar tabela de alertas e dar suporte às regras de detecção do Backend.

Tarefas

- **5.1** Criar tabela `alerts` :

```
1 CREATE TABLE alerts (  
2     id SERIAL PRIMARY KEY,  
3     host_id INT REFERENCES hosts(id) ON DELETE CASCADE,  
4     alert_type VARCHAR(50) NOT NULL, -- 'brute_force', 'port_scan'  
5     source_ip INET NOT NULL,  
6     timestamp TIMESTAMP DEFAULT NOW(),  
7     severity VARCHAR(20) DEFAULT 'medium',  
8     resolved BOOLEAN DEFAULT FALSE,  
9     resolved_at TIMESTAMP  
10 );  
11
```

- **5.2** Criar índice para buscas por IP e status:

- `CREATE INDEX idx_alerts_source_ip_resolved ON alerts(source_ip, resolved);`

- **5.3** Auxiliar o Backend na lógica de inserção de alertas (evitar duplicatas de alertas ativos para o mesmo IP).

- **5.4** Criar query para listar alertas ativos:

```
1 SELECT * FROM alerts WHERE resolved = FALSE ORDER BY timestamp DESC;  
2
```

- **5.5** Testar a geração de alertas junto com a equipe de Backend.

Entregáveis da Semana

- Tabela `alerts` operacional.
- Backend conseguindo persistir e consultar alertas.

♦ Semana 6 | 26 mai – 01 jun ⚠ Apresentação Parcial 29/05

 **Meta da Semana:** Garantir que o banco está estável e otimizado para a apresentação parcial.

Tarefas

- **6.1** Revisar índices e ajustar conforme necessidade real (baseado nas queries que o Frontend mais faz).
- **6.2** Criar um script de **seed** para popular o banco com dados de exemplo (métricas, logs e alertas) para a demonstração.
- **6.3** Verificar tamanho do banco e garantir que o volume não está crescendo descontroladamente.
- **6.4** Testar a integração completa: Agente → Backend → Banco → Frontend.

- **6.5** Estar disponível durante a apresentação parcial para eventuais ajustes de banco.

✓ Entregáveis da Semana

- Banco de dados estável e com dados de exemplo.
 - Performance adequada para a demo.
-

♦ Semana 7 | 02 – 05 jun 🚀 MVP Deadline

🎯 **Meta da Semana:** Finalizar documentação, backup/restore e ajustes finos.

📋 Tarefas

- **7.1** Configurar **backup lógico** diário (ou sob demanda) usando `pg_dump` :
 - Criar script `backup.sh` que executa `docker exec db pg_dump -U app_user monitor_db > backup_$(date +%Y%m%d).sql`.
- **7.2** Testar o restore do backup em um banco vazio para garantir integridade.
- **7.3** Documentar o schema do banco (tabelas, colunas, tipos, relacionamentos) em um `README_DB.md` ou na documentação principal.
- **7.4** Incluir instruções de como acessar o banco para depuração:
 - `docker exec -it db psql -U app_user -d monitor_db`
- **7.5** Verificar se o banco está preparado para o pitch: dados de simulação prontos, performance ok.

✓ Entregáveis da Semana

- Procedimento de backup e restore documentado e testado.
 - Documentação completa do banco de dados.
-

🎤 Período de Pitch (06 – 12 jun)

Tarefas do Banco de Dados para o Pitch

- **P.1** Garantir que o banco está populado com dados realistas para a demonstração.
 - **P.2** Apoiar a equipe caso surja alguma dúvida sobre consultas ou estrutura durante a apresentação.
 - **P.3** Estar pronta para explicar a modelagem e as decisões de arquitetura do banco.
-

⚠ Observações Técnicas

1. Persistência com Docker:

O volume mapeado (`./data/postgres`) é **obrigatório**. Sem ele, ao reiniciar o container todos os dados são perdidos. Adicione esta pasta ao `.gitignore`.

2. Senhas e Segredos:

Nunca commitar senhas no código. Use variáveis de ambiente (`.env`) para `POSTGRES_PASSWORD` e outras credenciais.

3. Performance com JSONB:

O campo `parsed_data` permite consultas flexíveis. Para alta performance em campos específicos (ex.: IP), considere criar colunas dedicadas no futuro, mas para o MVP o JSONB é suficiente.

4. Limpeza Automática:

A política de retenção pode ser implementada via `pg_cron` (extensão do PostgreSQL) ou via um script Python executado periodicamente (ex.: pelo próprio backend). O mais simples é um script Python agendado no host ou dentro de um container separado.

5. Ferramentas de Acesso:

Recomendo usar **DBeaver** ou **Beekeeper Studio** para conectar ao PostgreSQL durante o desenvolvimento (porta 5432 exposta apenas localmente).